AnchorMem: Intent-Aware Multi-Relational Evidence Graph Retrieval with Distilled Skills and Insights for Self-Evolving Agents

Anonymous Authors[1]

Abstract

Self-evolving LLM agents often face challenges due to underdeveloped memory systems that entangle multi-dimensional information or rely on raw historical context. We introduce **AnchorMem**, a comprehensive memory framework that enhances long-term reasoning and interactive decision-making by seamlessly coupling both Cross-Task and In-Task memory in a closed loop. Unlike traditional methods that rely on history replay, AnchorMem distills raw interactions into high-level Insight and Skill Graphs, enabling efficient, query-intent-aware retrieval through adaptive traversal of a Multi-Relational Evidence Graph. Extensive experiments on multiple benchmarks using the GPT-4o-mini and GPT-4.1-mini backbones demonstrate that AnchorMem significantly improves performance in long-horizon reasoning tasks and decision-making, even under zero-shot and one-trial settings. The results surpass state-of-the-art baselines, showcasing the framework's effectiveness and efficiency across a wide variety of tasks. Code is available at https://github.com/dhaiwodhaowih/AnchorMem

1. Introduction

Typically, an LLM agent is an interactive system that ingests observations from the environment, maintains context across multiple turns of interaction, and emits executable actions to accomplish a given task (Hu et al., 2025; Yao et al., 2023). However, constrained by fixed context windows, existing agents exhibit limitations in long-context, multi-turn settings, such as *context decay* and *losing track in the middle of interactions* (Liu et al., 2023; Wang et al., 2024a; Liu et al., 2025; Hu et al., 2025; Tu et al., 2025).

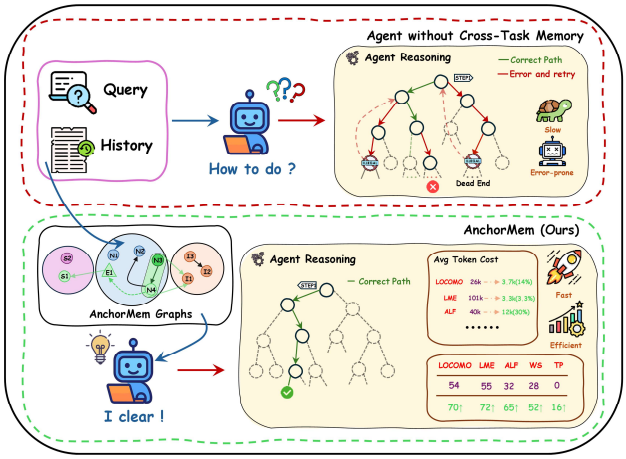


Figure 1. **AnchorMem Improves Agent Decision-Making Quality.** AnchorMem enhances long-horizon reasoning and facilitates self-evolution by leveraging Multi-Relational Evidence Graphs and cross-task abstractions for efficient decision-making, surpassing the limitations of semantic-similarity retrieval. As shown in the lower right, AnchorMem achieves significant improvements in both performance and efficiency across five benchmarks, including LOCOMO, LongMemEval, and ALFWorld (Wang et al., 2024b; Li et al., 2024; Shridhar et al., 2021); see Section 4 for details.

Consequently, LLM agents require stable external memory systems to retain past interactions and progressively build and track world knowledge.

What constitutes the ideal agent memory loop? An agent's memory can be divided into cross-task memory and in-task memory (long-term and working memory, respectively (Hu et al., 2025)). In-task memory captures transient execution traces, while cross-task memory stores distilled knowledge. Despite task diversity, many share structural similarities and similar solutions. In an ideal architecture, these systems work in synergy: the agent retrieves high-level priors from cross-task memory to constrain in-task search, while distilling task experiences into insights to refine cross-task memory. This interplay helps improve decision-making in long-horizon tasks (Wang et al., 2024a).

However, the ideal agent memory loop is often constrained by the structural and operational limitations of existing memory systems, making it difficult to realize in practice. Most

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

existing approaches store past interactions in a single repository or a minimally structured memory buffer, relying primarily on semantic similarity, temporal recency, or heuristic scoring for retrieval. For instance, G-Memory(Zhang et al., 2025) condenses historical information into a three-layer graph structure of "insight-query-trajectory," successfully refining cross-task memory at a coarse level. However, it primarily focuses on reusing abstract knowledge and overlooks the extraction of reusable trajectories, with its retrieval process relying solely on semantic embedding similarity, while neglecting temporal or causal relationships. Similarly, Nemori (Nan et al., 2025), a cognitively inspired memory system, optimizes around principled narrative segmentation and representation alignment, enabling effective detection of event boundaries and the construction of higher-level semantic summaries. However, its retrieval is confined to semantics, failing to distinguish complex events and leading to irrelevant responses. The lack of explicit modeling for multiple relationships and reusable abstract knowledge is the main limitation of current agents, hindering their ability for self-evolution.

We propose **AnchorMem**, a structured memory architecture inspired by human cognition. It distills cross-task interactions into reusable anchor-based knowledge and performs intent-aware retrieval over a Multi-Relational Evidence Graph (MREG). The MREG organizes evidence in a four-view structure (semantic, temporal, causal, and entity), while the Abstraction Graph (AG) distills insights and compresses action sequences into skill templates. AnchorMem ensures efficient, intent-aligned retrieval for long-horizon decision making, overcoming existing limitations in current memory systems (Hu et al., 2025; Tulving & Thomson, 1973).

AnchorMem couples in-task and cross-task memory in a closed loop: during execution, it records verifiable evidence into the MREG and distills it into transferable cross-task abstractions; conversely, retrieved abstractions serve as priors that constrain in-task retrieval and decision making. The MREG encodes objective world facts and agent experiences as a disentangled four-view graph, supporting intent-aligned evidence aggregation with traceable reasoning. The AG distills knowledge as insights and compresses recurring action structures into skill templates, providing compact priors that can be reused in subsequent tasks. The adaptive anchor diffusion retrieval strategy ensures efficient, dynamic selection and expansion of relevant abstractions during inference.

Our contributions are summarized as follows:

1. **Cross-task and in-task memory loop for agent self-evolution.** We propose AnchorMem, a structured memory system that integrates in-task experience into a Multi-Relational Evidence Graph (MREG) and distills cross-task knowledge into reusable memory anchors as *skills* and *insights* within the Abstraction Graph (AG). This closed-loop *write–retrieve–refine* process facilitates agent self-evolution, overcoming the limitations of existing methods by explicitly modeling multiple relationships and reusing abstract knowledge across tasks.

2. **Intent-aware, multi-relational retrieval through anchor seeding and expansion.** AnchorMem performs intent-conditioned anchor selection, seeding a compact frontier in the MREG, followed by adaptive expansion across relation-specific edges. This method ensures efficient, high-precision evidence aggregation aligned with the query intent, addressing the shortcomings of prior approaches that rely heavily on semantic similarity and fail to capture the full complexity of task-specific, temporal, and causal relationships.

3. **Strong empirical improvements on long-context reasoning and long-horizon interactive decision making tasks.** Using GPT-4o-mini, AnchorMem improves long-context reasoning, raising LLM-as-judge scores from 54.7% to 68.9% on LOCOMO and from 54.2% to 83.5% on LongMemEval. It outperforms strong memory methods such as SimpleMem(Liu et al., 2026), EMem(Zhou & Han, 2025), and Nemori(Nan et al., 2025). In decision-making tasks, AnchorMem increases success rates on ALFWorld from 32.4% to 63.3%, WebShop from 28.5% to 53.2%, and achieves a 15.55% final pass rate on TravelPlanner(Xie et al., 2024), where ReAct(Yao et al., 2023) fails completely.

Feature	MemGPT	LangMem	Zep	A-Mem	Mem0	EMem	G-Memory	SimpleMem	Nemori	AnchorMem (Ours)
Temporal reasoning	✗	✗	✓	✗	✗	✗	✗	✗	✓	✓
Causal reasoning	✗	✗	✗	✗	✗	✗	✓	✗	✗	✓
Entity-aware graph	✗	✗	✓	✓	✓	✓	✗	✗	✗	✓
Cross-task distillation	✗	✓	✗	✗	✗	✗	*partial*	✗	✗	✓
Procedural memory	✗	✓	✗	✗	✗	✗	✗	✗	✗	✓
Multi-relational retrieval	✗	✗	✓	✗	✗	✗	✗	✗	✗	✓
Intent-aware routing	✗	✗	✗	✗	✗	✗	✗	✗	✗	✓

Table 1. **Comparison of memory architectures. ✓: feature present; ✗: absent.** More discussion on existing memory architectures can be found in Appendix A and Appendix B.1.

2. Preliminary

In this section, we establish the notation and formalize key concepts of Self-evolving LLM-based agent memory and AnchorMem's hierarchical architecture.

2.1. Self-evolving LLM-based agent memory

Continual memory retrieval and updates are central to self-evolving agents. We adopt the Self-evolving LLM-based agent memory abstraction as a retrieve–generate–update loop (Gutiérrez et al., 2025). At interaction step t, the agent receives a query q_t and maintains an external memory state $\mathcal{M}_t$. The memory module retrieves context c_t, the LLM produces an action a_t, and the environment returns feedback r_t. We denote the observed outcome by $o_t \triangleq (a_t, r_t)$.

$$\begin{aligned} c_t &= \textsc{Retrieve}(q_t, \mathcal{M}_t), \\ a_t &= \text{LLM}(q_t, c_t), \\ \mathcal{M}_{t+1} &= \textsc{Update}(\mathcal{M}_t, q_t, a_t, r_t). \end{aligned} \tag{1}$$

AnchorMem instantiates $\mathcal{M}_t$ as a two-part memory consisting of a Multi-Relational Evidence Graph and an Abstraction Graph.

2.2. Multi-Relational Evidence Graph ($\mathcal{G}_e$)

Inspired by recent graph-based and hybrid retrieval-augmented generation systems (Edge et al., 2024; Latimer et al., 2025; Sarmah et al., 2024), we introduce the Multi-Relational Evidence Graph ($\mathcal{G}_e$). As the in-task substrate of AnchorMem, $\mathcal{G}_e$ maintains a typed, incrementally evolving record of fine-grained and verifiable execution evidence. Formally, we model $\mathcal{G}_e$ as

$$\mathcal{G}_e = (\mathcal{N}_e, \mathcal{E}_e), \tag{2}$$

where $\mathcal{N}_e$ denotes evidence nodes (e.g., observations, actions, tool outputs, and feedback) and $\mathcal{E}_e$ denotes typed relation edges connecting them. By explicitly structuring evidence and its relations, $\mathcal{G}_e$ supports intent-aligned subgraph retrieval with traceable provenance and enables faithful memory updates under environmental feedback.

Node definition. Each node $n \in \mathcal{N}_e$ is encoded as a tuple

$$n \triangleq \langle \kappa(n),\ s(n),\ v(n),\ t(n),\ A(n) \rangle, \tag{3}$$

where $\kappa(n) \in \{\texttt{event}, \texttt{entity}\}$ denotes the node kind; $s(n)$ is a concise textual summary; $v(n) \in \mathbb{R}^d$ is an embedding for indexing; $t(n)$ is an anchored timestamp (or step index); and $A(n)$ is a typed attribute set capturing metadata and provenance.

Edge definition. We define four typed edge spaces:

$$\mathcal{E}_e \ = \ \mathcal{E}_e^{\text{temp}} \ \cup \ \mathcal{E}_e^{\text{caus}} \ \cup \ \mathcal{E}_e^{\text{sem}} \ \cup \ \mathcal{E}_e^{\text{ent}}. \tag{4}$$

1. ***Temporal***($\mathcal{E}_e^{\text{temp}}$) forms an immutable temporal backbone: $(n_i, n_j) \in \mathcal{E}_e^{\text{temp}}$ if $\kappa(n_i) = \kappa(n_j) = \texttt{event}$ and $t(n_j) = t(n_i) + 1$.
2. ***Causal***($\mathcal{E}_e^{\text{caus}}$) encodes directed support among events for intent-conditioned "why" reasoning: $(n_i, n_j) \in \mathcal{E}_e^{\text{caus}}$ if $\kappa(n_i) = \kappa(n_j) = \texttt{event}$ and a consolidation score $S(n_i \to n_j)$ exceeds a threshold δ.
3. ***Semantic***($\mathcal{E}_e^{\text{sem}}$) connects conceptually related memory items: $(n_i, n_j) \in \mathcal{E}_e^{\text{sem}}$ if $\cos(v(n_i), v(n_j)) > \theta_{\text{sim}}$.
4. ***Entity***($\mathcal{E}_e^{\text{ent}}$) anchors events to entities with stable identifiers: $(n_i, n_j) \in \mathcal{E}_e^{\text{ent}}$ if $\kappa(n_i) = \texttt{event}$, $\kappa(n_j) = \texttt{entity}$, and n_i mentions or grounds n_j.

2.3. Abstraction Graph ($\mathcal{G}_a$)

AnchorMem maintains an Abstraction Graph, denoted by $\mathcal{G}_a$, to store cross-task abstractions distilled from in-task evidence. It comprises an *Insight Graph* and a *Skill Graph*:

$$\mathcal{G}_a \ = \ \mathcal{G}_i \ \cup \ \mathcal{G}_s. \tag{5}$$

All nodes in $\mathcal{G}_i$ and $\mathcal{G}_s$ are maintained under a unified Bayesian decision rule driven by posterior updates from execution feedback.

Insight Graph ($\mathcal{G}_i$). The Insight Graph stores transferable, high-level knowledge distilled from in-task evidence to steer future decisions. Each insight node $\text{Insight}_i \in \mathcal{G}_i$ is represented as

$$\text{Insight}_i \triangleq \langle s_i,\ \phi_i,\ v_i,\ e_i,\ \mathcal{S}_i,\ \mathcal{F}_i \rangle, \tag{6}$$

where fields s_i and v_i follow Eq. (3), ϕ_i specifies the prerequisite condition under which the insight is activated, and $e_i \subseteq \mathcal{N}_e$ is a supporting evidence set. We further maintain outcome logs $\mathcal{S}_i$ and $\mathcal{F}_i$ that record, respectively, the number of times the insight is validated or contradicted under execution feedback, together with the corresponding feedback instances. The edge space of $\mathcal{G}_i$ contains (i) ***implication edges*** between insight nodes, where $\text{Insight}_a \to \text{Insight}_b$ indicates that Insight_b is derivable from Insight_a, and (ii) ***evidence hyperedges*** that connect an insight node to its support set $e_i \subseteq \mathcal{N}_e$, grounding Insight_i in verifiable in-task evidence (Zhou et al., 2006).

Skill Graph ($\mathcal{G}_s$). The Skill Graph stores reusable procedural abstractions—compact action templates that can be invoked to guide future task execution. Each skill node $\text{Skill}_j \in \mathcal{G}_s$ is represented as

$$\text{Skill}_j \triangleq \langle s_j,\ a_j^{1:T},\ \phi_j,\ \psi_j,\ v_j,\ e_j,\ \mathcal{S}_j,\ \mathcal{F}_j \rangle, \tag{7}$$

where the shared fields $(s_j,\ v_j,\ e_j,\ \mathcal{S}_j,\ \mathcal{F}_j)$ follow Eq. (6). The condition pair (ϕ_j, ψ_j) specifies the start and termination criteria for invoking and ending the skill, and

$a_j^{1:T} = [a_j^1, \ldots, a_j^T]$ denotes an ordered action sequence. The edge space of $\mathcal{G}_s$ contains (i) ***containment edges*** forming a directed inclusion relation among skills (a parent skill subsumes a child's action sequence), and (ii) ***evidence hyperedges*** identical to those in $\mathcal{G}_i$, grounding each skill in its supporting nodes from $\mathcal{N}_e$.

Bayesian maintenance. A unified Bayesian decision process governs all nodes in $\mathcal{G}_i$ and $\mathcal{G}_s$, with the posterior-update rule specified in Section 4.

3. AnchorMem Framework

3.1. Initialization

AnchorMem does not require an explicit offline BUILD stage. Instead, we initialize $\mathcal{G}_e \triangleq (\mathcal{N}_e, \mathcal{E}_e)$ with $\mathcal{N}_e = \emptyset$ and $\mathcal{E}_e = \emptyset$, and populate it online: after each interaction step,UPDATE materializes newly observed, grounded evidence (observations/tool outputs, realized actions, and feedback) into typed nodes and multi-relational edges. Optional preloading of external grounded data can be treated as a special case of UPDATE with a synthetic feedback signal, which we omit for clarity.

3.2. Intent-Aware Multi-Relational Retrieval

As illustrated in Fig. 2, AnchorMem performs multi-strategy, intent-conditioned graph traversal rather than a static lookup over past traces. A query *coordinator* converts the query q into structured control signals and executes a multi-stage retrieval procedure (Algorithm 1), which dynamically selects relation views, traverses and aggregates evidence from $\mathcal{G}_e$, and, when beneficial, incorporates priors retrieved from $\mathcal{G}_a$.

Step 1: Query Analysis. We extract semantic, lexical, and temporal cues from q, producing three auxiliary control signals:

1. **Intent typing.** We infer an intent label $T_q \in \{\text{WHY}, \text{WHEN}, \text{ENTITY}\}$ via a lightweight classifier (or an LLM tagger), and translate it into relation-level traversal weights (e.g., WHEN upweights temporal edges).

2. **Distillation gating.** We predict whether q implies interactive execution; if positive, retrieval may consult the Skill Graph $\mathcal{G}_s$ as procedural priors, and the invoked skills become eligible for targeted posterior updates under feedback.

3. **Representation alignment.** We resolve relative time expressions into an absolute window $[\tau_s, \tau_e]$, encode q as a dense vector $\mathbf{q}$ for semantic indexing, and extract keywords q_{key} for BM25-style lexical retrieval (Robertson & Zaragoza, 2009).

Step 2: Adaptive Anchor Seeding. We first identify a small set of intent-aligned *anchors* as the seed frontier, and then expand from them to assemble a task-relevant evidence subgraph.

$$\mathcal{N}_{\text{anchor}} = \text{Top-}k_{\text{anchor}}\left(\sum_{m\in\{\text{sim},\text{bm25},\text{time}\}} \frac{1}{k_{\text{anchor}} + r_m(n)}\right), \tag{8}$$

RRF improves anchor recall under query variability.(Cormack et al., 2009)

Step 3: Anchor-Expansion Retrieval. Starting from the anchor frontier $\mathcal{N}_{\text{anchor}}$, we perform a budgeted best-first expansion on $\mathcal{G}_e$ to aggregate evidence aligned with the query intent.

We using an intent-conditioned transition score:

$$S(n_j \mid n_i, q) = \exp\Big(\lambda_1 \mathbf{w}_{T_q}^\top \mathbf{e}_{ij} + \lambda_2 \,\text{sim}(n_j, q)\Big), \tag{9}$$

where $\mathbf{w}_{T_q}$ is the relation-weight vector induced by the query intent T_q, and $\mathbf{e}_{ij}$ is the one-hot encoding of the edge type for e_{ij}. We define semantic affinity by cosine similarity:

$$\text{sim}(n_j, q) = \frac{v(n_j)^\top \mathbf{q}}{\|v(n_j)\|_2 \,\|\mathbf{q}\|_2}, \tag{10}$$

We maintain a priority queue over frontier nodes by their accumulated scores and iteratively expand the highest-scoring candidates until the node budget is reached. To avoid drift, we enforce (i) a maximum hop depth from $\mathcal{N}_{\text{anchor}}$ and (ii) a time-window constraint when $[\tau_s, \tau_e]$ is available. The resulting visited set induces a compact subgraph $\mathcal{G}_e^{\text{sub}}$ that preserves both semantic relevance and relation-aligned structure.

Step 4: Abstraction retrieval. We retrieve cross-task priors from $\mathcal{G}_a$ by selecting abstractions that are (i) applicable to the current context, (ii) supported by the retrieved anchors, and (iii) high-utility under execution feedback. Concretely, for each abstraction node $\mathrm{a}_i \in \mathcal{G}_a$ with start condition ϕ_i and support set $e_i \subseteq \mathcal{N}_e$, we compute an expected-utility score $\text{EU}(\mathrm{a}_i \mid c_t)$ (Appendix C gives the full instantiation). We then aggregate anchor support and select abstractions whose aggregated utility exceeds a threshold, keeping the top-$K_{\mathcal{A}}$:

$$\mathcal{A}_t = \text{Top-}k_{\mathcal{A}}\left(\left\{\sum_{n\in\mathcal{N}_{\text{anchor}}\cap e_i} u(\mathrm{a}_i, c_t, n)\,\text{EU}(\mathrm{a}_i \mid c_t)\right\}\right)$$

$$u(\mathrm{a}_i, c_t, n) = \begin{cases} 1, & \text{EU}(\mathrm{a}_i \mid c_t) \geq \tau, \\ 0, & \text{otherwise.} \end{cases} \tag{11}$$

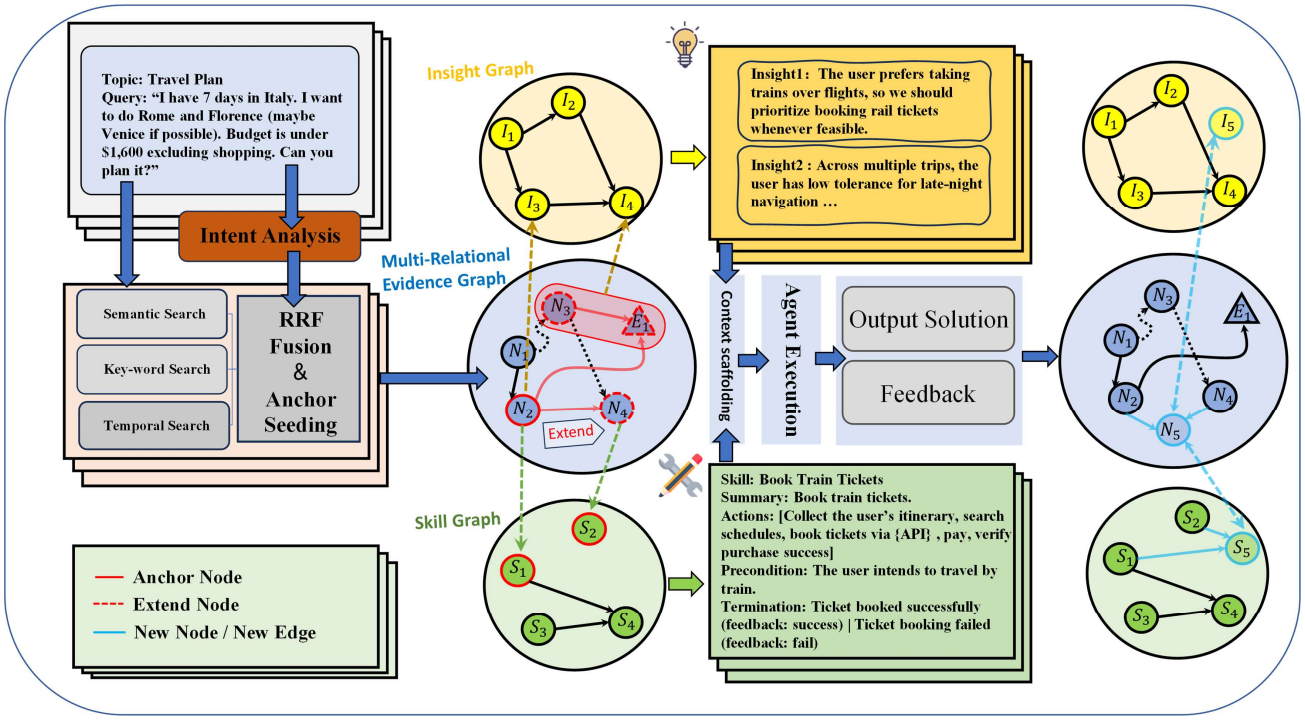


Figure 2. **AnchorMem Workflow.** The diagram illustrates the integration of multi-relational evidence retrieval, anchor-expansion retrieval , abstraction retrieval and adaptive memory updating in AnchorMem.

Intuitively, an abstraction is favored if it is applicable and repeatedly supported by the current anchors, while low-utility abstractions are filtered out by the threshold τ.

Step 5: Context scaffolding with provenance. We linearize retrieved evidence with explicit provenance and prepend selected abstractions:

$$c_t = \left(\bigoplus_{\mathrm{a}_i \in \mathcal{A}_t} P(\mathrm{a}_i) \right) \oplus \left(\bigoplus_{n \in \pi(\mathcal{G}_e^{\mathrm{sub}}, T_q)} B(n) \right) \tag{12}$$

where $t(n)$ is the timestamp of evidence node n, and $s(n)$ is its textual summary. We let $\pi(\mathcal{G}_e^{\mathrm{sub}}, T_q)$ order nodes by query intent (e.g., chronological for WHEN.)

3.3. Anchored Memory Update and Maintenance

Memory evolution couples in-task evidence accumulation in $\mathcal{G}_e$ with cross-task abstraction refinement in $\mathcal{G}_a$, forming a closed loop that updates both what the agent knows and how it acts after each episode; we realize it via three phases at the evidence, abstraction, and maintenance levels.

Evidence Anchoring. At the evidence level, we update $\mathcal{G}_e$ by instantiating a verified node and linking it only within an intent-induced local neighborhood:

$$\begin{aligned} n_{\mathrm{new}} &\leftarrow \textsc{Extract}(t) \\ \mathcal{N}_{\mathrm{conn}} &\leftarrow \textsc{LocalNbr}(n_{\mathrm{new}}, \mathcal{G}_e^{\mathrm{old}}, I_t) \\ \mathcal{E}_{\mathrm{new}} &\leftarrow \{(n, n_{\mathrm{new}}) \mid n \in \mathcal{N}_{\mathrm{conn}}\} \\ \mathcal{G}_e^{\mathrm{new}} &\leftarrow \langle \mathcal{N}_e^{\mathrm{old}} \cup \{n_{\mathrm{new}}\}\ \mathcal{E}_e^{\mathrm{old}} \cup \mathcal{E}_{\mathrm{new}} \rangle \end{aligned} \tag{13}$$

$\textsc{LocalNbr}(\cdot)$ restricts candidates to recent anchors and explicitly supported semantic/entity links; causal links are inserted only under unambiguous Resolved/Failed feedback to avoid spurious attribution.

Cross-Task Distillation (Anchoring). At the abstraction level, we update $\mathcal{G}_a$ in two ways: (i) refining utilized abstractions under success/failure logs, and (ii) instantiating new abstractions when the episode provides sufficient evidence for transfer. For a utilized abstraction a (insight or skill), we contrast its successful and failed applications and update its

Algorithm 1 Intent-Aware Multi-Relational Retrieval

Input: query q, $\mathcal{G}_e, \mathcal{G}_a$
Output: narrative context c
// Step 1: Query Analysis (intent typing, gating, alignment)
$\langle T_q, \mathbf{q}, q_{\text{key}}, [\tau_s, \tau_e], g\rangle \leftarrow \textsc{AnalyzeQuery}(q)$
$\mathbf{w}_{T_q} \leftarrow \textsc{GetWeights}(T_q)$
// Step 2: Anchor seeding (hybrid retrieval + RRF)
$\mathcal{N}_{\text{anchor}} \leftarrow \text{RRF-}\textsc{Anchor}(\mathbf{q}, q_{\text{key}}, [\tau_s, \tau_e])$
// Step 3: Anchor-guided expansion on $\mathcal{G}_e$
$\mathcal{G}_e^{\text{sub}} \leftarrow \textsc{TraverseEG}(\mathcal{G}_e,\ \mathcal{N}_{\text{anchor}},\ \mathbf{w}_{T_q},\ [\tau_s, \tau_e])$
// Step 4: Abstraction-graph traversal on $\mathcal{G}_a$ (EU with anchor support)
if $g = 1$ **then**
    $\mathcal{A}_t \leftarrow \textsc{SelectAbstractions}(\mathcal{G}_a,\ \mathcal{N}_{\text{anchor}},\ q)$
else
    $\mathcal{A}_t \leftarrow \emptyset$
end if
// Step 5: Context scaffolding with provenance
$c \leftarrow \textsc{Scaffold}(\mathcal{A}_t,\ \mathcal{G}_e^{\text{sub}},\ T_q)$
return c
Transition score used in TraverseEG:
$S(n_j \mid n_i, q) = \exp\Big(\lambda_1\, \mathbf{w}_{T_q}^{\top} \mathbf{e}_{ij} + \lambda_2\, \text{sim}(n_j, q)\Big)$

start/end conditions and (if applicable) its action sequence:

$$\begin{aligned}
\mathcal{D}_a &\leftarrow \textsc{Contrast}(\mathcal{S}_a, \mathcal{F}_a) \\
\phi_a &\leftarrow \phi_a \cup \Delta\phi_a^{+} \cup \{\neg\varphi \mid \varphi \in \Delta\phi_a^{-}\} \\
\psi_a &\leftarrow \psi_a \cup \Delta\psi_a \cdot \mathbf{1}[\text{type}(a) = \texttt{skill}] \\
a_a^{1:T} &\leftarrow \textsc{Merge}(a_a^{1:T}, \Delta a_a^{1:T}) \cdot \mathbf{1}[\text{type}(a) = \texttt{skill}]
\end{aligned} \tag{14}$$

We instantiate a new insight only when a piece of evidence is repeatedly retrieved and consistently succeeds; in this case, we anchor it with its historical neighborhood in $\mathcal{G}_e$ and distill a concise, reusable statement. We instantiate a new skill only when a RESOLVED episode yields a complete minimal action trace, which is distilled into a reusable procedural anchor for future episodes.

Memory Management. To prevent unbounded growth of $\mathcal{G}_a$, we maintain fixed budgets for insights and skills and periodically keep only the top entries by a multi-factor utility score:

$$\begin{aligned}
U(a) \;= \lambda_r \cdot \frac{\alpha_a}{\alpha_a + \beta_a} \;+\; \lambda_f \cdot \frac{n_a}{\sum_{a'} n_{a'}} \\
+\; \exp\left(-\frac{t - t_a^{\text{last}}}{\gamma}\right)
\end{aligned} \tag{15}$$

Here $\alpha_a/(\alpha_a + \beta_a)$ is reliability under execution feedback, n_a is usage frequency, and t_a^{last} is the last-used time. Overall, this budgeted pruning retains abstractions that are both frequently used and reliably successful, while evicting those that are rarely invoked and consistently ineffective. We provide additional sensitivity results in Appendix C for choosing $(\lambda_r, \lambda_f, \lambda_t)$.

Algorithm 2 Anchored Memory Update and Maintenance

Input: episode trace t, intent I_t, feedback y, $\mathcal{G}_e, \mathcal{G}_a$
Output: updated $\mathcal{G}_e, \mathcal{G}_a$
// Evidence Anchoring
$n_{\text{new}} \leftarrow \textsc{Extract}(t)$
$\mathcal{N}_{\text{conn}} \leftarrow \textsc{LocalNbr}(n_{\text{new}}, \mathcal{G}_e, I_t)$
$\mathcal{E}_{\text{new}} \leftarrow \{(n, n_{\text{new}}) \mid n \in \mathcal{N}_{\text{conn}}\}$
$\mathcal{G}_e \leftarrow \textsc{UpdateEG}(\mathcal{G}_e, n_{\text{new}}, \mathcal{E}_{\text{new}}, y)$ // causal only if y is unambiguous
// Cross-Task Distillation (Anchoring)
$\mathcal{A}_{\text{used}} \leftarrow \textsc{UsedAbs}(t, \mathcal{G}_a)$
$\mathcal{G}_a \leftarrow \textsc{RefineAbs}(\mathcal{G}_a, \mathcal{A}_{\text{used}}, t)$ // uses Contrast/Merge
$\mathcal{G}_a \leftarrow \textsc{InstantiateAbs}(\mathcal{G}_a, \mathcal{G}_e, t, y)$
// Memory Management
$U(a) \leftarrow \lambda_r \frac{\alpha_a}{\alpha_a+\beta_a} + \lambda_f \frac{n_a}{\sum_{a'} n_{a'}} + \exp\big(-\frac{t-t_a^{\text{last}}}{\gamma}\big)$
$\mathcal{G}_a \leftarrow \textsc{BudgetPrune}(\mathcal{G}_a, U)$
return $\mathcal{G}_e, \mathcal{G}_a$

4. Experiment

In this section, we evaluate AnchorMem on long-memory reasoning and decision-making tasks to demonstrate how its integration of Cross-Task and In-Task memory improves performance and outperforms state-of-the-art baselines across relevant benchmarks.

4.1. Experiment Setup.

Benchmarks. To validate AnchorMem's integration of Cross-Task and In-Task memory, we use long-memory recall benchmarks (e.g., LOCOMO (Wang et al., 2024b), LongMemEval (Li et al., 2024)) and long-horizon interactive decision-making tasks (e.g., ALFWorld (Shridhar et al., 2021), WebShop (Yao et al., 2022), TravelPlanner (Xie et al., 2024)). These benchmarks highlight the need for memory systems supporting both task-specific retrieval and cross-task knowledge transfer. More details are in Appendix B.1.

Baselines. We compare AnchorMem against SimpleMem, Mem0 (Chhikara et al., 2025), A-Mem (Xu et al., 2025), Full-Context, EMem (Zhou & Han, 2025), LangMem (Liu et al., 2024), Nemori, Zep (Rasmussen et al., 2025) for long-memory benchmarks, and ReAct, Voyager, and G-Memory for interactive benchmarks. These baselines highlight the need for a more efficient memory architecture, motivating AnchorMem's development. More details are in Appendix B.2.

Backbone	Method	SingleHop			MultiHop			Temporal			OpenDomain			Macro			Token
		LLM Score	F1	BLEU	LLM Score	F1	BLEU	LLM Score	F1	BLEU	LLM Score	F1	BLEU	LLM Score	F1	BLEU	
GPT-4o-mini	Full-Context	83.00±2.4	27.15	26.32	64.80±2.6	17.13	14.19	56.20±2.8	26.67	24.26	47.60±3.0	24.82	18.65	54.75±2.7	23.94	19.48	26070
	LangMem	66.12±1.8	34.80	31.10	49.31±2.0	28.50	23.90	26.10±2.2	31.90	26.20	52.56±2.4	29.40	23.50	47.29±2.2	30.09	24.61	406
	A-Mem	40.09±1.9	27.02	20.09	35.31±2.1	12.14	12.00	54.16±2.3	45.85	36.67	48.16±2.5	44.65	37.06	46.55±2.3	37.85	31.33	2897
	Mem0	67.42±1.2	36.72	25.13	55.17±1.4	26.39	27.00	51.76±1.3	46.93	38.51	45.83±1.6	37.12	35.34	50.12±1.5	37.18	33.84	1837
	Zep	62.15±1.1	34.44	22.67	51.66±1.3	19.37	14.82	49.12±1.4	42.00	34.53	41.20±1.5	41.15	36.56	46.07±1.4	36.92	31.29	3916
	EMem	72.12±0.9	47.77	39.44	56.30±1.1	33.46	23.81	66.05±1.2	52.55	46.41	58.20±1.4	25.29	19.26	60.36±1.3	33.87	27.01	1231
	Nemori	77.60±1.0	50.40	41.20	65.30±1.2	30.20	23.60	66.76±0.8	51.72	44.19	44.70±1.3	47.28	40.52	55.12±1.2	45.27	38.23	3786
	SimpleMem	40.11±1.1	35.89	32.74	69.50±1.3	33.98	25.74	59.60±1.2	52.14	43.02	61.20±1.4	31.89	30.79	61.07±1.3	36.74	32.54	3375
	AnchorMem	**62.06**±0.8	**33.09**	**23.34**	**68.30**±0.9	**28.02**	**25.16**	**66.00**±1.0	**47.86**	**37.73**	**72.90**±0.6	**48.69**	**41.66**	**69.94**±0.7	**43.76**	**36.68**	**4080**
GPT-4.1-mini	Full-Context	84.60±2.5	50.40	41.60	75.90±2.7	36.50	22.60	74.20±2.9	47.50	40.00	55.10±3.2	28.40	22.20	64.73±3.0	35.24	27.32	26070
	LangMem	84.40±1.9	51.00	43.30	69.70±2.1	41.40	31.90	52.30±2.3	48.50	39.60	56.60±2.5	31.90	25.50	59.84±2.3	38.29	30.71	414
	A-Mem	45.77±2.0	41.02	36.99	38.65±2.2	25.06	17.32	55.19±2.4	51.01	44.75	56.11±2.6	13.22	14.75	52.08±2.4	24.99	22.84	3165
	Mem0	71.42±1.2	39.20	32.60	34.99±1.4	34.30	25.20	56.76±1.3	44.40	37.60	53.86±1.6	22.70	17.70	52.10±1.5	30.38	24.16	1810
	Zep	60.21±1.0	30.50	30.50	60.05±1.2	30.50	20.40	79.90±0.9	23.80	20.00	51.00±1.4	24.10	19.20	59.26±1.2	25.61	20.29	3876
	EMem	82.90±0.9	49.90	40.50	70.30±1.1	31.10	26.20	77.00±1.0	49.10	38.90	66.30±1.3	26.50	26.80	70.30±1.2	33.51	29.35	1165
	Nemori	84.90±0.8	57.20	53.10	74.86±1.0	41.80	31.90	78.30±0.9	57.30	50.50	51.40±1.2	25.80	19.30	63.39±1.1	37.25	30.13	3702
	SimpleMem	46.60±1.1	43.46	39.60	73.80±1.3	40.46	36.82	62.60±1.2	53.15	47.67	65.10±1.4	19.77	18.00	65.02±1.3	31.99	28.71	3216
	AnchorMem	**71.06**±0.7	**41.89**	**38.15**	**75.30**±0.9	**37.58**	**35.11**	**73.70**±1.0	**52.55**	**48.68**	**74.60**±0.6	**34.68**	**28.51**	**73.75**±0.7	**39.39**	**34.42**	**3764**

Table 2. Performance on the LoCoMo benchmark evaluated using F1, BLEU1, LLM-as-a-Judge Score, and Avg Token Cost as metrics. Dark blue indicates the best performance, while light blue indicates the second best.

Evaluation. LLM-as-a-Judge score is used for long-memory benchmarks, and Success Rate (SR) for interactive benchmarks. Additional metrics include token-level F1, BLEU-1 for LOCOMO, and FP for TravelPlanner.

4.2. Long-Memory Reasoning Benchmarks

Tables 2, 5, and Fig. 3 compare representative memory architectures on long-memory reasoning benchmarks under two LLM backbones.

Evaluation Scores. As shown in Table 2, on GPT-4o-mini, AnchorMem achieves the highest LLM score of 68.90, outperforming Full-Context (54.75), A-Mem (46.55), and SimpleMem (61.07) by 12% to 48%. On GPT-4.1-mini, AnchorMem scores 71.87, maintaining a 2.2% to 38% advantage, and over 10% when excluding EMem. On LongMemEval, AnchorMem achieves the highest score of 72.33, surpassing the next best baseline by 17%. This demonstrates the effectiveness of AnchorMem's multi-graph, multi-relation retrieval structure for long-memory reasoning.

Token Consumption. AnchorMem uses only 4k tokens on LOCOMO, compared to Full-Context's 26.07k tokens, saving 84%. On LongMemEval, AnchorMem achieves 130% of Full-Context's performance with just 3.2% of the token usage. Combining Tables 2 and 5, we observe that AnchorMem achieves the highest accuracy with moderate token consumption, effectively compressing long interaction histories into compact subgraphs while retaining key information.

Highlights Analysis. AnchorMem excels in Open-Domain and Knowledge-Update tasks, achieving Open-Domain scores of 72.9 and 74.6 , surpassing competitors by up to 31.7 . In Knowledge-Update, it scores 88.67, leading by up to 27.3 points. These results demonstrate AnchorMem's highly generalized memory architecture and its ability to handle complex, long-context in-task memory.

4.3. Long-Horizon Interactive Benchmark Analysis

Method	ALF (SR)	WB (SR)	TP (FP)
ReAct	32.08	28.5	0
Voyager	52.24	43.6	13.88
GMemory	56.71	46.6	9.4
AnchorMem	**64.93**	**53.2**	**15.55**

Table 3. **Performance comparison on long-horizon interactive benchmarks.** ALF,WB,TP : ALFWorld, Webshop, TravelPlanner. SR: validation success rate; FP: validation final pass rate (end-to-end completion). All results use GPT-4o-mini as the backbone. The results are evaluated under zero-shot + 1 trial settings.

As shown in Table 3, AnchorMem outperforms all competitors across all three interactive benchmarks. On ALFWorld, AnchorMem surpasses ReAct, Voyager, and GMemory by 102.9%, 24.3%, and 14.4%, respectively. On WebShop and TravelPlanner, this margin increases to 86.1%, 22.1%, 14.2% and infinity , 12.1%, and 65.4%, respectively. While both Voyager and GMemory demonstrate preliminary cross-task knowledge transfer capabilities, they lack a multi-perspective, multi-view memory architecture and do not explicitly distill insights and skills. This results in their performance being inferior to that of AnchorMem, highlighting the superiority of our architecture.

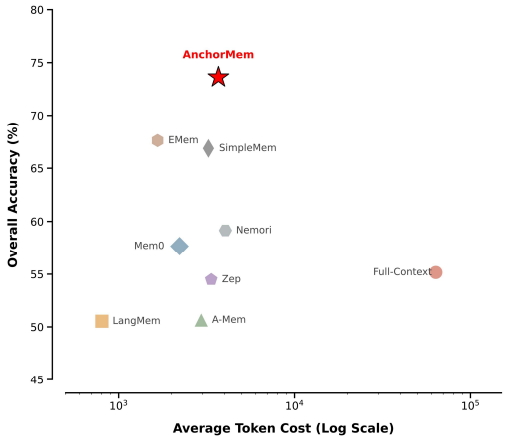


Figure 3. **Accuracy–Cost Trade-off.** Integrated comparison of memory baselines on long-memory reasoning benchmarks, showing overall accuracy versus average token cost (log scale).

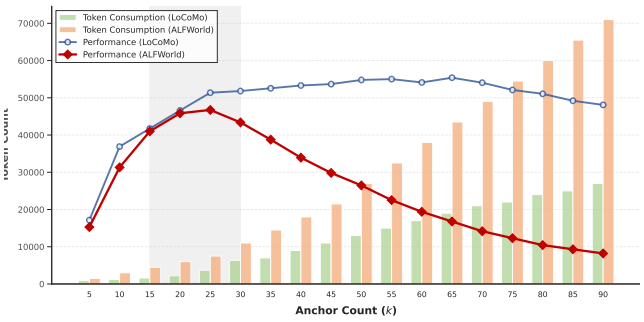
Ablated Component	LoCoMo Score	LoCoMo %	ALFWorld SR	ALFWorld %
Full (**AnchorMem**)	**73.62**	**100.0**	**62.69**	**100.0**
w/o Abstraction Graph ($\mathcal{G}_a$)	69.95	95.01	51.95	83.40
w/o Intent-Aware Retrieval	67.44	91.60	59.70	95.23
w/o Intent Decomposition	67.68	91.93	60.45	96.42
w/o Bayesian Maintenance	70.23	95.40	53.73	85.71

Table 4. **Core ablations of AnchorMem.** The results show that removing $\mathcal{G}_a$ and Bayesian Maintenance causes significant performance drop, especially on ALF, highlighting their crucial roles in AnchorMem and dynamic maintenance for long-horizon tasks.

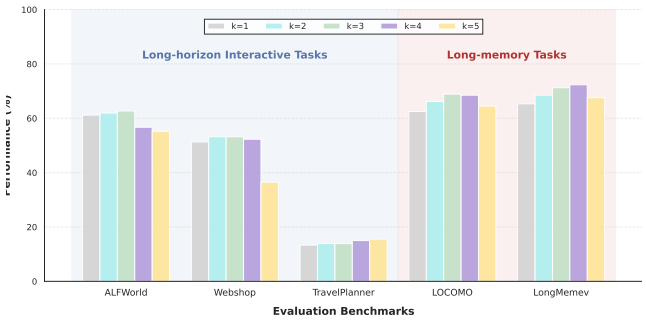
4.4. Ablation and Superparameter Analysis

Ablation. Table 4 presents the results of ablation studies on key components of AnchorMem. Removing $\mathcal{G}_a$ reduces performance by 5.67% and drops the success rate by 10.29%. $\mathcal{G}_a$ is essential for distilling cross-task knowledge, enabling the agent to adapt to new tasks. Removing Bayesian Maintenance further lowers performance, highlighting the need for dynamic memory updates. Without it, the agent's knowledge becomes outdated, leading to suboptimal decision-making. Intent-Aware Retrieval ensures precise retrieval based on task-specific intent. Its removal causes the agent to retrieve irrelevant information, hurting decision-making accuracy. Finally, Intent Decomposition breaks tasks into sub-goals for efficient retrieval. Without it, the agent loses the ability to prioritize and efficiently use memory, leading to inefficient decision-making. These results underscore the importance of combining all components for efficient long-term reasoning and decision-making, overcoming traditional memory system limitations. Removing key components such as $\mathcal{G}_a$ or Bayesian Maintenance leads to significant performance drops, underscoring their importance in cross-task knowledge distillation and dynamic memory updates. IAR and ID further optimize memory retrieval and task prioritization, with their absence causing retrieval inefficiencies and reduced decision-making accuracy.

Superparameter Analysis. As shown in Figure 4(a), the optimal k_{anchor} range is 15 to 30, balancing retrieval and token cost. Figure 4(b) shows k_{anchor} sensitivity varies by task type. For long-text memory tasks, performance peaks at $k_{\text{anchor}} \in \{3, 4\}$; for simpler tasks, $k_{\text{anchor}} \in \{1, 2\}$ is optimal. In complex tasks, higher k_{anchor} is more effective. Similarly, the optimal $k_{\mathcal{A}}$ for abstract node retrieval varies by task complexity. For long-text tasks, $k_{\mathcal{A}} \in \{3, 4\}$ works best, while for simpler tasks, $k_{\mathcal{A}} \in \{1, 2\}$ reduces unnecessary abstractions. These results highlight the need for adaptive hyperparameter tuning to optimize performance across tasks.



((a)) Sensitivity analysis of k_{anchor}.



((b)) Sensitivity analysis of $k_{\mathcal{A}}$.

Figure 4. (a). k_{anchor} in Equation 8, controls the number of anchors retrieved for memory recall; (b). $k_{\mathcal{A}}$ in Equation 3.2, determines the number of abstract nodes retrieved for task abstraction. All the experiments here are done with GPT-4o-mini.

References

Chhikara, P. et al. Mem0: Building production-ready ai agents with scalable long-term memory, 2025.

Cormack, G. V., Clarke, C. L. A., and Buettcher, S. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR)*, 2009.

Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., Truitt, S., Metropolitansky, D., Ness, R. O., and Larson, J. From local to global: A graph rag approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024.

Gutiérrez, B. J., Shu, Y., Qi, W., Zhou, S., and Su, Y. From rag to memory: Non-parametric continual learning for large language models. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025. URL https://arxiv.org/abs/2502.14802.

Hu, Y., Liu, S., Yue, Y., Zhang, G., Liu, B., Zhu, F., Lin, J., Guo, H., Dou, S., Xi, Z., Jin, S., Tan, J., Yin, Y., Liu, J., Zhang, Z., Sun, Z., Zhu, Y., Sun, H., Peng, B., Cheng, Z., Fan, X., Guo, J., Yu, X., Zhou, Z., Hu, Z., Huo, J., Wang, J., Niu, Y., Wang, Y., Yin, Z., Hu, X., Liao, Y., Li, Q., Wang, K., Zhou, W., Liu, Y., Cheng, D., Zhang, Q., Gui, T., Pan, S., Zhang, Y., Torr, P., Dou, Z., Wen, J.-R., Huang, X., Jiang, Y.-G., and Yan, S. Memory in the age of ai agents, 2025.

Latimer, C., Boschi, N., Neeser, A., Bartholomew, C., Srivastava, G., Wang, X., and Ramakrishnan, N. Hindsight is 20/20: Building agent memory that retains, recalls, and reflects. *arXiv preprint arXiv:2512.12818*, 2025.

Li, K. et al. Longmemeval: Benchmarking long-term memory in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

Liu, J., Xiong, K., Xia, P., Zhou, Y., Ji, H., Feng, L., Han, S., Ding, M., and Yao, H. Agent0-vl: Exploring self-evolving agent for tool-integrated vision-language reasoning, 2025.

Liu, J., Su, Y., Xia, P., Han, S., Zheng, Z., Xie, C., Ding, M., and Yao, H. Simplemem: Efficient lifelong memory for llm agents. *arXiv preprint arXiv:2601.02553*, 2026. URL https://arxiv.org/abs/2601.02553. Preprint.

Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., and Liang, P. Lost in the middle: How language models use long contexts, 2023.

Liu, S., Hu, Y., Yue, Y., et al. Langmem: Long-term language model memory. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2024.

Nan, J., Ma, W., Wu, W., and Chen, Y. Nemori: Self-organizing agent memory inspired by cognitive science. *arXiv preprint*, 2025.

Packer, C., Fang, V., Patil, S., Gonzalez, J. E., Wooders, S., and Stoica, I. Memgpt: Towards lifelong memory for llms. In *Proceedings of the Conference on Neural Information Processing Systems (NeurIPS)*, 2023.

Rasmussen, P., Paliychuk, P., Beauvais, T., Ryan, J., and Chalef, D. Zep: A temporal knowledge graph architecture for agent memory, 2025.

Robertson, S. and Zaragoza, H. The probabilistic relevance framework: Bm25 and beyond. *Foundations and Trends® in Information Retrieval*, 3(4):333–389, 2009.

Sarmah, B., Hall, B., Rao, R., Patel, S., Pasquali, S., and Mehta, D. Hybridrag: Integrating knowledge graphs and vector retrieval augmented generation for efficient information extraction. *arXiv preprint arXiv:2408.04948*, 2024.

Shridhar, M. et al. Alfworld: Aligning text and embodied environments for interactive learning. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021.

Tu, A., Xuan, W., Qi, H., Huang, X., Zeng, Q., Talaei, S., Xiao, Y., Xia, P., Tang, X., Zhuang, Y., et al. Position: The hidden costs and measurement gaps of reinforcement learning with verifiable rewards, 2025.

Tulving, E. and Thomson, D. M. Encoding specificity and retrieval processes in episodic memory. *Psychological Review*, 80(5):352–373, 1973.

Wang, T., Tao, M., Fang, R., Wang, H., Wang, S., Jiang, Y. E., and Zhou, W. Ai persona: Towards life-long personalization of llms, 2024a.

Wang, Z. et al. Locomo: Evaluating long-term conversational memory in llm-based agents. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024b.

Xie, J., Zhang, K., Chen, J., Zhu, T., Lou, R., Tian, Y., Xiao, Y., and Su, Y. Travelplanner: A benchmark for real-world planning with language agents. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024. doi: 10.48550/arXiv.2402.01622. Spotlight.

Xu, W., Liang, Z., Mei, K., Gao, H., Tan, J., and Zhang, Y. A-mem: Agentic memory for llm agents, 2025.

Yao, S., Chen, H., Yang, J., and Narasimhan, K. Webshop: Towards scalable real-world web interaction with grounded language agents. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. arXiv:2207.01206.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.

Zhang, G., Fu, M., Wan, G., Yu, M., Wang, K., and Yan, S. G-memory: Tracing hierarchical memory for multi-agent systems, 2025.

Zhou, D., Huang, J., and Schölkopf, B. Learning with hypergraphs: Clustering, classification, and embedding. In *Advances in Neural Information Processing Systems*, volume 19, pp. 1601–1608. MIT Press, 2006.

Zhou, S. and Han, J. A simple yet strong baseline for long-term conversational memory of llm agents. *arXiv preprint*, 2025.

A. Related Work

The memory mechanism is a cornerstone for autonomous agents to achieve long-term planning and personalized interaction. Current research in agent memory can be broadly categorized into unstructured memory, semantic-structured memory, and procedural memory.

A.1. Unstructured Memory

Unstructured memory treats conversation history or external knowledge as a continuous stream of text, often managed through sliding windows or summarization.

- **SimpleMem**: As a baseline implementation, it relies on linear storage and direct context filling. While it is easy to deploy and computationally efficient, it suffers from severe context dilution and "lost in the middle" issues during long-term tasks (Packer et al., 2023).
- **Zep**: It excels in engineering efficiency by providing fast, asynchronous storage and automatic dialogue summarization. However, its compression logic often discards nuanced details, making it less effective for tasks requiring precise retrieval of specific past facts (Rasmussen et al., 2025).
- **MemGPT**: Inspired by operating systems, it introduces tiered memory (main memory and external storage) managed by explicit LLM function calls. Its strength lies in overcoming hard context limits through active management, but it imposes high computational overhead and latency due to the iterative self-editing of memory (Packer et al., 2023).

A.2. Semantic-Structured Memory

These models utilize embeddings, vector databases, or graphs to organize information based on semantic relevance, enabling more precise retrieval.

- **EMem (Enhanced Memory)**: It utilizes attention-based mechanisms to enhance the extraction of critical information from high-density history. While it improves retrieval precision, it struggles with the high dimensionality of long-term embeddings, which can lead to semantic drift (Zhou & Han, 2025).
- **Mem0**: It provides a user-centric memory layer that learns preferences across sessions. Its main advantage is cross-session consistency; however, its incremental updating mechanism can occasionally lead to "memory conflicts" where outdated and new information coexist (Chhikara et al., 2025).
- **G-Memory (Graph-based Memory)**: By organizing memory into a graph structure, it captures complex topological relations between entities. While superior for multi-hop reasoning, it faces significant challenges in graph maintenance and the scalability of real-time graph construction (Zhang et al., 2025).
- **Nemori**: This framework mimics biological memory through "forgetting" and "consolidation" cycles. It effectively prunes noise but relies on complex heuristic thresholds that are difficult to tune for different domains (Nan et al., 2025).

A.3. Procedural Memory

Procedural memory focuses on "how" to perform tasks, storing action sequences, skills, or Standard Operating Procedures (SOPs).

- **LangMem**: Developed by the LangChain team, it optimizes agent prompts based on long-term observation of user behavior. It is excellent for prompt engineering automation but lacks the flexibility to handle rapidly changing task environments (Liu et al., 2024).
- **A-Mem (Action-oriented Memory)**: It transforms successful operation trajectories into a reusable skill library. While it significantly reduces reasoning costs for repetitive tasks, it lacks generalization capabilities when the agent encounters entirely novel scenarios not covered by the skill bank (Xu et al., 2025).

A.4. Summary and Comparison

While unstructured models like *MemGPT* offer maximum flexibility, they are often resource-intensive. Structured approaches like *G-Memory* and *Mem0* provide better logical consistency but require complex indexing. Procedural systems like *A-Mem* represent the trend toward experience-driven control, though they still struggle with zero-shot adaptation. Modern agents increasingly move toward hybrid architectures that integrate these three types of memory to balance efficiency and reasoning depth.

B. Details of benchmarks and baselines.

B.1. Details of benchmarks.

In this section, we describe the five benchmark suites used to evaluate long-term memory, reasoning, and planning capabilities of language agents. For each benchmark, we summarize the evaluation dimensions, task types, and task counts (where publicly available). Our presentation emphasizes concrete task definitions and dataset compositions to aid reproducibility and reviewer understanding.

B.1.1. LoCoMo: Long-Term Conversational Memory Benchmark

Benchmark Overview. LoCoMo (*Long-term Conversational Memory*) evaluates persistent memory and reasoning over extended multi-session dialogues produced by LLM-based agent architectures. The benchmark was introduced with a machine–human pipeline generating dialogues spanning up to dozens of sessions and thousands of tokens, with explicit persona and temporal event graph grounding.
Evaluation Dimensions.

- **Question Answering (QA):** Measures recall and reasoning over highly extended conversational history.
- **Event Graph Summarization:** Assesses temporal and causal inference by summarizing event structures derived from dialogues.
- **Multi-Modal Dialogue Generation:** Evaluates generation of contextually consistent responses across modalities (text and images).

Task Types and Numbers. LoCoMo's primary QA task categorizes queries into multiple reasoning types, including:

- *Single-hop reasoning*, factual recall from one session.
- *Multi-hop reasoning*, chaining information across sessions.
- *Temporal reasoning*, inferring order or timing of events.
- *Open-domain*, requiring integration of worldview knowledge.

The QA dataset comprises approximately 1,986 questions distributed across these categories, with per-category counts such as 282 multi-hop and 321 temporal items (other categories summing to the total) in typical benchmark releases. Event summarization and multi-modal generation add further diagnostic evaluations, though these do not have unified "task counts" analogous to QA items.

B.1.2. LongMemEval: Long-Term Memory Evaluation Benchmark

Benchmark Overview. LongMemEval evaluates sustained interactive memory capabilities of chat assistants and memory-augmented language systems. It embeds evaluation questions within long, extensible user–assistant histories to challenge models' abilities to retain, update, and reason over information distributed across many sessions.

Evaluation Dimensions.

- **Information Extraction:** Identifying salient facts across extended dialogues.
- **Multi-Session Reasoning:** Combining information across sessions to answer queries.
- **Temporal Reasoning:** Understanding chronological relationships.
- **Knowledge Updates:** Handling changes in information over time.
- **Abstention:** Recognizing unanswerable queries or irrelevant memory.

Task Types and Numbers. LongMemEval consists of 500 curated questions, each associated with a long, timestamped chat history. Questions are designed to probe the five long-term memory abilities listed above, making this set a concise yet challenging evaluation suite.

B.1.3. ALFWorld: Textual and Embodied Agent Benchmark

Benchmark Overview. ALFWorld is a multimodal environment benchmark that connects abstract text reasoning with embodied decision-making. Agents first learn high-level symbolic policies in a text-based environment (TextWorld) and then execute them in a visual, embodied simulation based on the ALFRED environment.

Evaluation Dimensions.

- **Navigation and Interaction:** Assessing the agent's ability to interact with objects and environments.
- **Language Grounding:** Evaluating how well text-based planning transfers to embodied execution.
- **Cross-Modal Generalization:** Testing whether learned policies in symbolic/textual space generalize to visually rich settings.

Task Types and Numbers. ALFWorld includes several thousand distinct task instances spanning abstract text tasks and their embodied visual counterparts. Published implementations (e.g., training and held-out evaluation cases) often refer to on the order of 3,553 training tasks and 134 unseen evaluation tasks for specific agent models, though exact counts vary by environment configuration.

B.1.4. WebShop: Web Interaction and Decision-Making Benchmark

Benchmark Overview. WebShop is a large-scale simulated e-commerce environment for evaluating web navigation, grounding, and decision-making by autonomous language

agents. In this benchmark, agents receive natural language purchase instructions and must navigate a simulated web interface with multiple page types to locate, customize, and purchase products. **Evaluation Dimensions.**

- **Instruction Understanding:** Interpreting item specifications in free text.
- **Web Navigation:** Performing actions across web pages to satisfy the instruction.
- **Strategic Exploration:** Efficiently exploring noisy page content and structural diversity.

Task Types and Numbers. WebShop's dataset consists of 12,087 crowd-sourced text instructions paired with a simulated catalog of 1.18 million real-world products. Each instruction defines a navigation and selection task in the environment. Agents are evaluated on success rate and cumulative reward across these tasks.

B.1.5. TravelPlanner: Real-World Planning Benchmark

Benchmark Overview. TravelPlanner is a planning benchmark grounded in real-world travel itinerary generation. It provides a rich sandbox environment with nearly four million data records accessed via diverse tools and evaluates an agent's capability to construct feasible multi-day travel plans under complex user constraints.

Evaluation Dimensions.

- **Tool Use and Information Gathering:** Evaluating whether agents can effectively gather and integrate information from external tools (e.g., city search, flights, lodging).
- **Constraint Satisfaction:** Assessing compliance with explicit user constraints (e.g., budget, room type) and implicit commonsense rules.
- **Coherent Plan Generation:** Measuring the feasibility and consistency of the full plan across multiple days.

Task Types and Numbers. TravelPlanner comprises 1,225 curated travel planning queries. The queries are grouped by travel duration and the number of hard constraints. The dataset is split into training (45 annotated plans), validation (180 queries), and a test set of 1,000 queries. Each query requires generating a full itinerary that satisfies both environmental and user needs.

B.2. Details of Baselines

In this section, we describe the baseline memory systems referenced in this work. For each baseline, we summarize its core technical design, strengths, and weaknesses, and provide a concise comparison with AnchorMem's design philosophy and capabilities.

B.2.1. MemGPT (Operating System–Inspired Memory)

Core Idea. MemGPT introduces a hierarchical "virtual memory" framework inspired by classical operating systems, wherein an LLM actively manages its own memory by paging data between in-context and external storage. The agent leverages explicit function calls to move information in and out of the model's finite context window, enabling extended context retention without simply expanding the window size. In MemGPT, the memory hierarchy consists of a working context (analogous to RAM) and an external archive (analogous to disk) that stores conversational history outside the model's immediate context. Function chaining and self-editing enable multi-step retrieval and memory updates without external intervention. **Strengths.**

- Self-directed memory management via LLM functions avoids manually engineered retrieval heuristics; the model learns what to store and when to retrieve it.
- Flexible multi-session recall that can exceed native context window limits by dynamically swapping information.
- The underlying concept has been widely adopted in open-source agent frameworks (e.g., Letta) for persistent agent behavior.

  Weaknesses.

  – System complexity is high: correctly orchestrating multi-tier memory and function calls requires careful task-specific engineering.
  – Retrieval precision is moderate, particularly for benchmarks emphasizing temporal reasoning over multiple sessions.
  – Memory management can incur latency due to repeated function calls and context manipulation.

  Comparison with AnchorMem. While MemGPT emphasizes self-editing and context paging, AnchorMem focuses on structured memory representations and efficient retrieval via integrated memory hierarchies tailored for both reasoning depth and operational efficiency across multiple benchmarks.

B.2.2. Zep (Temporal Knowledge Graph Memory)

Core Idea. Zep constructs a temporally aware knowledge graph that synthesizes unstructured conversation history and structured data into a unified graph representation. Nodes represent entities or events, and edges

encode temporal and semantic relationships, enabling agents to reason over evolving facts and cross-session context. The underlying engine, named *Graphiti*, dynamically ingests new data and updates the graph structure for accurate long-term recall. This approach significantly enhances temporal reasoning compared to flat vector memory stores. **Strengths.**

- Graph structures enable explicit modeling of temporal relationships and multi-entity interactions.
- Demonstrates strong performance on temporal and cross-session reasoning benchmarks, including LongMemEval.
- Graph-based memory supports efficient incremental updates as new interaction data arrives.

Weaknesses.

- Graph maintenance and consistency can be computationally expensive at scale.
- Effectiveness depends on graph quality and entity extraction accuracy, which may degrade for noisy open-domain dialogue.
- Engineering complexity is higher than simpler vector retrieval baselines.

Comparison with AnchorMem. AnchorMem's memory structure similarly captures relational context but emphasizes modular retrieval and reasoning components that balance efficiency and expressivity. Zep's graph modeling excels at tracking evolution of entities, whereas AnchorMem's hybrid retrieval may offer better generalization across heterogeneous tasks.

B.2.3. MEM0 (SCALABLE EXTRACT-AND-RETRIEVE MEMORY)

Core Idea. Mem0 introduces a scalable memory architecture that dynamically extracts salient conversational facts and maintains them in a compact store for retrieval. A dual-phase pipeline—an extraction phase to identify important information and an update phase to manage memory operations (ADD, UPDATE, DELETE, NO-OP)—keeps memory coherent and non-redundant. A graph-enhanced variant (Mem0^{g}) overlays relation edges to facilitate richer multi-hop retrieval.

Strengths.

- Strong performance on long-term conversational reasoning tasks, including multi-hop and temporal queries on LoCoMo.
- Efficient retrieval pipeline with lower latency and reduced token cost relative to full-context baselines.
- Graph augmentation further improves relational reasoning while preserving scalability.

Weaknesses.

- Baseline performance reports show disparities across tasks when compared to optimized graph variants.
- Pure extraction pipelines may miss loosely connected facts not classified as salient at extraction time, reducing recall in some long-tail cases.

Comparison with AnchorMem. AnchorMem extends the scalable extraction paradigm with additional structural biases and procedural integration, enabling broader retrieval adaptability and improved memory consolidation across interactive benchmarks.

B.2.4. LANGMEM (VECTOR-INDEXED EPISODIC MEMORY)

Core Idea. LangMem is a vector-indexed memory layer that stores embeddings of interactions into a vector database. Retrieval operates via similarity search, followed by lightweight reranking, to identify relevant memory segments. This architecture prioritizes simplicity and alignment with semantic similarity measures for episodic recall.

Strengths.

- Conceptually simple and easy to integrate into existing agent pipelines.
- Vector search enables semantic match retrieval without explicit structure.
- Good baseline performance for single-hop recall tasks.

Weaknesses.

- High latency due to large vector scans for long histories.
- Underperforms on complex multi-hop and temporal reasoning compared to structured baselines.
- Lack of temporal modeling limits cross-session reasoning fidelity.

Comparison with AnchorMem. AnchorMem incorporates structured retrieval strategies that address LangMem's limitations on temporal and multi-hop reasoning while retaining the semantic richness that vector retrieval provides.

B.2.5. A-MEM (AGENTIC MEMORY NETWORKS)

Core Idea. A-Mem proposes an agentic memory system that dynamically organizes memories via interconnected notes, inspired by the Zettelkasten method. Each memory encodes multiple structured attributes (e.g., contextual descriptions, keywords, tags), and the system establishes links between memories based on

contextual similarity, enabling continuous evolution of the memory network as new experiences arise. index=8

Strengths.

- Memory notes capture rich semantic context beyond raw text storage.
- Dynamic linking supports memory evolution, potentially improving recall consistency over long sequences.
- Demonstrates generalization improvements against baseline memory systems across multiple foundational models.

Weaknesses.

- Memory organization relies on complex linking strategies, which may be sensitive to noise or inconsistent embeddings.
- Engineering overhead is higher due to structured note creation and maintenance.

Comparison with AnchorMem. While A-Mem focuses on rich note linking, AnchorMem balances structured retrieval with task-aware indexing to maintain scalability across diverse benchmark tasks. AnchorMem's memory mechanisms integrate task relevance and retrieval performance more directly.

C. Detailed Instantiation of Abstraction Retrieval

C.1. Abstraction-Graph Traversal and Expected Utility Scoring

In this section, we provide the full instantiation of the abstraction retrieval procedure referred to in Step 4 of Section 4, focusing on the instantiation of abstraction-graph traversal and utility scoring. Our approach integrates Bayesian evaluation of abstraction reliability, contextual similarity scoring, and anchor-conditioned utility aggregation. This method draws inspiration from hierarchical procedural memory selection mechanisms, such as those implemented in MACLA, where Bayesian posteriors over reliability scores guide the evaluation of expected utility .

Bayesian Reliability Modeling. Each abstraction node a_i, whether representing an insight or a skill, maintains a record of past application outcomes, summarized by success and failure counts, denoted as (α_i, β_i). The reliability of each abstraction is modeled using a Beta distribution as a conjugate prior, with the posterior distribution over the latent correctness parameter ρ_i being:

$$p(\rho_i \mid \mathcal{D}_i) = \mathrm{Beta}(\rho_i\,;\, \alpha_i, \beta_i),$$

where $\mathcal{D}_i$ represents the feedback history aggregated for abstraction a_i. This Bayesian framework ensures that the reliability of each abstraction adapts as more feedback becomes available, offering a probabilistic measure of how likely a given abstraction is to be correct.

Contextual Expected Utility. The expected utility of an abstraction, conditioned on the current context, is central to the retrieval process. Let o_t denote the current retrieval context, which consists of the original query and the evidence summary retrieved from the Multi-Relational Evidence Graph $\mathcal{G}_e$. For each abstraction node a_i, we compute its expected utility as follows:

$$\begin{aligned}\mathrm{EU}(\mathrm{a}_i \mid o_t) = \mathbf{1}[\phi_i(o_t) = 1] \cdot \mathrm{sim}(o_t, \mathrm{a}_i) \cdot \frac{\alpha_i}{\alpha_i + \beta_i} R_{\max} \\ - \mathrm{Risk}_i(o_t) \cdot \frac{\beta_i}{\alpha_i + \beta_i} C_{\text{fail}} \\ + \lambda_{\text{info}} \cdot H[\mathrm{Beta}(\alpha_i, \beta_i)],\end{aligned}$$

where:

- $\phi_i(o_t)$ is a Boolean predicate indicating whether the abstraction is applicable to the current context, - $\mathrm{sim}(o_t, \mathrm{a}_i)$ is a similarity measure between the context and the abstraction, - $R_{\max}$ and C_{fail} represent the maximum reward and failure cost, respectively, - $\mathrm{Risk}_i(o_t)$ estimates the empirical failure rate of abstraction a_i for contexts similar to o_t, - $H[\cdot]$ denotes the differential entropy of the Beta distribution, promoting the selection of abstractions with high information content, - λ_{info} is a weighting factor for the information gain term.

This formulation balances several key factors: contextual relevance, abstraction reliability, risk, and information gain. The expected utility framework ensures that abstractions are selected not only based on their direct relevance but also on their reliability and the risk of failure in the given context, following Bayesian selection principles.

Anchor-Conditioned Utility Aggregation. Each abstraction a_i is linked to a set of supporting evidence, denoted as $e_i \subseteq \mathcal{N}_e$, via hyperedges in the Abstraction Graph. Given a set of anchors $\mathcal{N}_{\text{anchor}}$ identified during retrieval, the utility of a_i for each supporting anchor evidence $n \in \mathcal{N}_{\text{anchor}} \cap e_i$ is computed as:

$$\mathrm{EU}(\mathrm{a}_i \mid o_t, n) = \mathrm{EU}(\mathrm{a}_i \mid o_t) \cdot \kappa(n),$$

where $\kappa(n)$ is a scaling factor that adjusts the utility based on the quality or recency of the anchor evidence,

typically measured as a normalized similarity between anchor evidence n and the context o_t.

The aggregated utility score for each abstraction a_i is then computed by summing the utilities of all supporting anchors. A threshold function $u(\cdot)$ is applied to retain only positive utility contributions, ensuring that only those abstractions which offer tangible benefits are considered:

$$u(\mathrm{a}_i, c_t, n) = \begin{cases} 1, & \text{if } \mathrm{EU}(\mathrm{a}_i \mid o_t, n) \geq \tau, \\ 0, & \text{otherwise.} \end{cases}$$

The final aggregated utility for abstraction a_i is:

$$U_{\mathrm{agg}}(\mathrm{a}_i) = \sum_{n \in \mathcal{N}_{\mathrm{anchor}} \cap e_i} u(\mathrm{a}_i, c_t, n) \cdot \mathrm{EU}(\mathrm{a}_i \mid o_t, n).$$

Top-K Abstraction Selection. To select the most relevant abstractions, we apply a Top-K selection criterion, which retains the $K_{\mathcal{A}}$ abstractions with the highest aggregated utility. This ensures that only abstractions with the highest expected benefit under the available evidence are selected:

$$\mathcal{A}_t = \text{Top-}K_{\mathcal{A}} \left(\{ U_{\mathrm{agg}}(\mathrm{a}_i) \mid \mathrm{a}_i \in \mathcal{G}_a \} \right).$$

This method guarantees that the selected abstractions are both contextually relevant and supported by strong evidence, offering the highest expected benefit in the current context.

Computational Considerations. Computing the expected utility for all nodes in the Abstraction Graph $\mathcal{G}_a$ can be computationally expensive. To optimize this process, we limit the candidate set to a pre-filtered subset of abstractions, based on coarse semantic indexing and anchor adjacency. This significantly reduces the computational load and overall retrieval latency.

—

C.2. Sensitivity Analysis of Bayesian Memory Maintenance Hyperparameters

We conducted a structured sensitivity analysis to investigate the impact of the Bayesian memory maintenance hyperparameters on the pruning utility score:

$$U(a) = a \cdot \frac{\alpha_a}{\alpha_a + \beta_a} + b \cdot \frac{n_a}{\sum_{a'} n_{a'}} + c \cdot \exp\left(-\frac{t - t_a^{\mathrm{last}}}{\gamma} \right),$$

where a, b, and c control the influence of posterior reliability, usage frequency, and temporal recency on the retained abstractions in the Abstraction Graph $\mathcal{G}_a$. These parameters are constrained by the condition $a + b + c = 1$.

Our grid search explored various combinations of these hyperparameters, including:

- $a \in \{0.2, 0.4, 0.6, 0.8\}$,
- $b \in \{0.1, 0.2, 0.3\}$,
- $c = 1 - a - b$ (ensuring $a + b + c = 1$ and $c \geq 0$).

Through this analysis, we found that the best performing configuration across benchmarks was $(a, b, c) = (0.6, 0.2, 0.2)$.

This sensitivity analysis demonstrated that increasing the reliability weight (a) consistently improved performance across both the LOCOMO (macro LLM-as-judge score) and ALFWorld (success rate) benchmarks. Conversely, high recency emphasis (c) alone led to suboptimal performance, highlighting the importance of balancing reliability, frequency, and recency for effective memory maintenance.

The selected combination of parameters provides a robust foundation for AnchorMem's performance, ensuring stability across different benchmarks without being overly sensitive to small perturbations in the hyperparameters.

D. Supplementary Experiment Results

D.1. Performance on LongMemeval Benchmark.

BB	Method	S-S-U	S-S-A	S-S-P	M-S	K-U	T-R	Macro	Token
(Mini)		Score	Score	Score	Score	Score	Score	Score	Cost
GPT-4o-mini	Full-Context	89.76	89.76	6.70	38.30	79.41	37.51	55.57	101k
	LangMem	70.10	60.50	41.70	58.00	75.60	28.10	53.79	1.2k
	A-Mem	78.25	47.26	30.00	52.30	72.50	43.11	54.74	3k
	Mem0	82.86	26.07	90.87	63.21	66.11	68.12	65.22	2.6k
	Zep	92.90	74.80	72.64	41.20	76.85	53.41	62.90	2.8k
	EMem	81.30	79.40	32.20	72.10	82.40	65.80	70.45	2.1k
	Nemori	88.60	81.90	46.70	51.10	61.30	58.80	63.18	4.3k
	SimpleMem	82.10	84.20	31.30	71.20	79.50	63.40	69.64	3.1k
	AnchorMem	**81.56**	**85.12**	**52.64**	**78.45**	**88.67**	**73.40**	**72.33**	**3.3k**

Table 5. Performance on LongMemeval benchmark (BB: Backbone). S-S-U: User queries; S-S-A: Assistant queries; S-S-P: Preference; M-S: Multi-Session; K-U: Knowledge Update; T-R: Temporal Reasoning.

E. Prompts

1. Evaluation Prompt: Semantic Fidelity Assessment

You are tasked with evaluating the semantic accuracy of a memory retrieval system's response. Rate the Candidate Answer relative to the Gold Reference on a continuous scale from [0.0 to 1.0].

Scoring Guidelines:

- **1.0 (Perfect Match):** The response accurately captures all important entities, temporal references, and causal relationships, matching the meaning of the Gold Reference.
- **0.8 (Mostly Correct):** The answer conveys the main idea but misses minor details or nuances.
- **0.6 (Partially Correct):** The response contains some valid information but omits critical elements (e.g., incorrect date but correct event).
- **0.4 (Off-Topic):** The response addresses the topic but lacks essential information or understanding.
- **0.2 (Incorrect):** The answer is factually inaccurate, with minimal overlap in content.
- **0.0 (Contradiction/Erroneous):** The answer is completely unrelated or contradicts the facts in the Gold Reference.

Evaluation Conditions:

- **Handling of Temporal Flexibility:** Accept answers that use relative time expressions (e.g., "next Monday") if they correspond to the same time frame as the Gold Reference.
- **Emphasis on Meaning:** Focus on the accuracy of the information conveyed, not just exact wording or phrasing.
- **Handling of Unanswerable Queries:** If the Gold Reference indicates "Unanswerable," the Candidate must explicitly state the absence of information. Any fabricated data results in a score of 0.0.

Input Format:

- **Question:** question
- **Gold Reference:** gold
- **Candidate Answer:** generated

Output Format (JSON):

```
{
  "score": float,
  "explanation": "brief reasoning behind the sco
}
```

E.1. Modified Query-Adaptive QA Prompt with Skills and Insights

This section introduces a modified version of the **Query-Adaptive QA Prompt** used within the **MAGMA** system, designed to utilize both **skills** and **insights** from the agent's memory. The modification allows the model to leverage learned skills for specific tasks and apply insights from prior interactions to make more informed decisions.

SYSTEM PROMPT:

System Role: You are a precision QA assistant operating on retrieved memory contexts. Your goal is to answer the user's question accurately using only the provided information, leveraging the skills and insights stored in the memory.
Context: context
Current Query:
• **Question:** question
• **Constraints:** category$_s pecific_c onstraints$
Instructions:

1. Use **ONLY information explicitly stated in the context**, as well as relevant skills and insights that apply to the current situation.
2. If the answer is not present in the context, respond exactly with "`Information not found.`"
3. Be **concise** (typically 1–10 words) unless detailed reasoning or explanation is required.
4. Use your **skills** and **insights** to make inferences where necessary:
   – **Skills:** If the question involves patterns or tasks you've learned, apply those directly.
   – **Insights:** Use prior knowledge from relevant experiences to guide reasoning or fill in logical gaps, as long as they align with the context.
5. $\{\texttt{dynamic}_i nstruction\}//Automaticallygeneratedbyourengine'squeryclassifier/router(nooraclelabels)$.

Answer:

Key Modifications:

The modified **QA Prompt** now incorporates the following elements:

– **Skills:** The model can apply learned patterns or task-related expertise to answer specific types of queries. This means the model can recognize recurring tasks and leverage its past learning to answer them more effectively.
– **Insights:** The model can make inferences or fill logical gaps based on insights it has gathered from past experiences. These insights help guide the reasoning process, particularly when the context is ambiguous or lacking explicit answers.
– **Concise Answers:** While being concise is emphasized, the model is allowed to provide more detailed responses if reasoning is required, making the answers adaptable to the complexity of the query.

F. Case Study

ALFWorld Case Study: "Put chilled lettuce on the counter"

Task Setup:

The task involves a two-phase goal: the lettuce must first be cooled before it can be placed on the counter. The term "chilled" specifies that cooling must be done prior to placement. This task is categorized as unseen since the specific combination of object-appliance-location (lettuce-fridge-countertop) was not encountered during training, thus testing the agent's ability for compositional generalization.

Initial State:

- Agent Location: In the kitchen.
- Objects: Lettuce is located on countertop 2; the fridge 1 is available but currently closed.

Memory State: The skill memory includes learned skills such as:

- object_cooling (success rate 76.9
- object_placement (success rate 80.0

The insight memory contains insights drawn from previous tasks, such as:

- Cooling Insight: The cooling skill is applicable to any task requiring cold preparation, regardless of the object, whether it's apples, potatoes, or other items.
- Placement Insight: Successful placement relies heavily on ensuring the object is both cooled and in hand, which is consistent across various tasks.

Meta-insight memory includes learned patterns from other object configurations (e.g., potato-fridge-table, apple-fridge-shelf), but no pre-existing skill directly matches the lettuce-fridge-countertop configuration.

Execution Timeline:

1. Initial Task Parsing (t0): - AnchorMem Parsing: The agent parses the task and identifies "chilled" as a necessary precondition. It establishes that the object must first be cooled before it is placed on the counter. The system identifies the cooling skill as the first sub-task and retrieves the object_cooling skill from memory.

2. Action 1 (t1 - t2): - Memory Retrieval: The agent retrieves the object_retrieval skill to pick up the lettuce from the counter. - Execution: The agent moves to countertop 2 and picks up the lettuce, confirming the action by updating the context.

3. Action 2 (t3 - t4): - Cooling Phase: The agent proceeds to the fridge for cooling. The Bayesian Selector evaluates fridge and freezer options, favoring the fridge based on a higher posterior success rate (E[] = 0.769). - Precondition Check: The agent verifies the feasibility of the cooling action by ensuring the precondition ¬cooled(lettuce 2) is met. - Execution: The agent opens the fridge (using a sub-skill) and initiates the cooling process for the lettuce.

4. Action 3 (t5 - t6): - Post-Cooling Check: The agent confirms that the lettuce is cooled (cooled(lettuce 2) = true). - Next Phase: The agent transitions to the placement task, retrieving the object_placement skill and moving to the countertop 2 for placement.

5. Action 4 (t7 - t8): - Placement Execution: The agent places the cooled lettuce on the counter, completing the task.

Post-Episode Learning and Insight:

After completing the task, the system logs both success and failure contexts to improve its skill memory. The following insight was drawn from the task:

Insight: The agent learned that the cooling precondition ("chilled") frequently appears in tasks involving refrigeration, but in unseen scenarios like this one with the lettuce, the task-specific configuration plays a significant role in determining the success of the skill. The agent found that when tasks involve new object-appliance-location combinations, it can enhance skill recall by generalizing across similar setups and linking relevant preconditions, even when exact examples were not seen during training.

This insight allowed the system to adapt its skill knowledge dynamically, recognizing that the cooling skill (even if not specific to lettuce) could be applied in any case where the precondition of "chilled" is required, regardless of the specific object. This deepened the agent's generalization ability, ensuring more efficient completion of similar tasks in the future.

Contrastive Refinement:

During contrastive refinement, the system detected latent discrepancies in the cooling skill that could be applied across similar tasks. Specifically, the system identified that "cooling" is relevant not only for lettuce but also for other objects requiring cold preparation. Thus, this skill was kept general to allow future reuse in various contexts.

The Bayesian posterior update (E[] for the object_cooling skill) reflects an increased confidence in its reliability after the successful completion of the task, improving the skill's utility for future retrievals.

Result:

The task was successfully completed, and AnchorMem's skill memory was updated, enabling it to more efficiently handle similar tasks in the future. By leveraging the insight about the generalization of object-specific actions (like cooling) across various contexts, AnchorMem achieved better compositional generalization in subsequent tasks, such as those involving different object-appliance combinations.

Conclusion:

In this case study, AnchorMem's ability to generalize across unseen configurations was enhanced by the system's insight into the reusable nature of skills. The integration of Bayesian decision-making and contrastive learning allowed the agent to perform tasks effectively even in unfamiliar contexts, by recognizing and applying learned patterns from similar tasks. This highlights the potential of hierarchical skill memory and insights in enabling efficient, adaptive learning in LLM-based agents.

G. Conclusion

Our core contribution is **AnchorMem**, a memory architecture that couples in-task and cross-task memory in a closed loop. By efficiently integrating **in-task evidence** (such as execution traces) with **cross-task abstractions** (such as reusable skills and insights), AnchorMem enables the agent to perform **self-evolution**. This approach supports long-horizon reasoning and decision-making tasks by retrieving relevant knowledge from past experiences and applying it to current tasks. The system introduces a **Multi-Relational Evidence Graph (MREG)** that structures the memory along four relational dimensions—semantic, temporal, causal, and entity—and utilizes **Bayesian updates** to adapt the system's knowledge over time. The agent can then use this integrated memory to handle complex decision-making tasks across various domains effectively.

H. Limitations

While **AnchorMem** demonstrates significant improvements over state-of-the-art systems, there are some areas for further enhancement. One limitation is the **complexity of maintaining the Multi-Relational Evidence Graph (MREG)**, which requires efficient graph traversal techniques and careful memory management, particularly as the number of stored interactions increases. Additionally, the system's performance may still be impacted by the quality of initial knowledge or **insights** if the memory architecture does not sufficiently generalize to all domains or task types. Moreover, although **self-evolution** is a promising concept, ensuring the system can dynamically adapt to new and unseen contexts without excessive reliance on prior knowledge remains an ongoing challenge.